

- Welcome to 711!
- Xinfeng Ye (week 1 to 6)
 - Room: 303-589
 - x.ye@auckland.ac.nz
- Sathiamoorthy Manoharan (week 7 to 12)
 - Room: 303.593
 - mano.manoharan@auckland.ac.nz

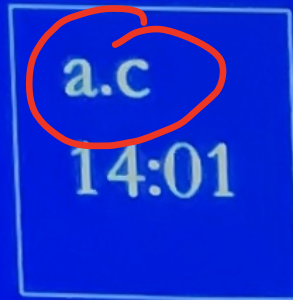
- Assignment 1:
 - weight: 15%
 - C#, socket programming
- Assignment 2:
 - weight: 15%
 - C#
- Test (in-class, in-person invigilated, 10/04)
 - Weight: 20%
- Exam (in-person invigilated):
 - weight: 50%

Logical time

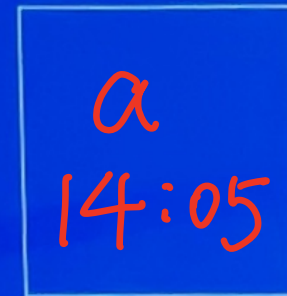
因果矢系

- Causality between events is fundamental to the design of parallel and distributed systems.
- In distributed systems, it hard to have global physical time.
 - It is only possible to achieve an approximation of it.
 - Machines might reside in different time zone.

edit



compile & run



edit

a.c
14:01

error

compile & run

a
14:05

edit

a.c
14:03
14:03

compile & run

a
14:05
14:05



make

error.

- For many applications in distributed systems, what usually matters is **not** that all tasks agree on **exactly** the **real time**, but rather they **agree on the order** in which events occur

Synchronising Logical Clocks

同步

concept.

■ happens before

- If a and b are **events** in the same process, and a occurs before b , then $a \rightarrow b$ is true.
- If a is the event of a **message** being sent by one process, and b is the event of the message being received by another process, then $a \rightarrow b$ is also true.
 - A message cannot be received before it is sent, or even at the same time that it is sent, since it takes a finite amount of time to arrive.



- Happen before is a **transitive** relation
 - if $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$
- Need a way to represent time such that for every event, say a , a **time value** $c(a)$ can be assigned on which all processes agree. These time values must have the property that **if $a \rightarrow b$, then $c(a) < c(b)$** .
- **Logical clock** is the device which is used to record the time for a process.
 - The time which is assigned to an event is based on the value of the logical clock.
 - The logical clock is a counter like variable.

Value
↑
time stamp

logical clock

```
loop forever
    send message
    if an event occurs in the process
    then assign the value of clock to the event;
        clock := clock + 1
    endif
    if a message m is received
    then clock := max(time(m), clock) + 1;
        assign the value of clock to the receiving
        message event
        clock := clock + 1
    endif
endloop
```

0

P1

P2

0

event 1

A (0)

event 2.

Send to P2 (1)

M(1)

event

C (2)

event
E (3)

after E

 $C = 4$

Receive from P2 (5)

clock:

 $\max(4, 4) + 1$ ~~clock~~ + 1 $\max(1, 0) + 1$

Receive from P1 (2)

event

D (3)

after .D

M(4)

Send to P1 (4)

 $C = 4$

- Happen before is a **transitive** relation
 - if $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$
- Need a way to represent time such that for every event, say a , a **time value** $c(a)$ can be assigned on which all processes agree. These time values must have the property that **if $a \rightarrow b$, then $c(a) < c(b)$** .
- **Logical clock** is the device which is used to record the time for a process.
 - The time which is assigned to an event is based on the value of the logical clock.
 - The logical clock is a counter like variable.

total order

- Some events in different processes **cannot be ordered** according to the time.
 - This is the case when the timestamps of two events from different processes are the same.
 - In order to be able to give an order in this situation, the identifier of the processes are used.
 - A timestamp is a pair **(time, process-identifier)**.
 - The identifiers of the processes are unique
- Assume that the timestamps for a and b are (T_a, Id_a) and (T_b, Id_b) respectively.
 - $a \rightarrow b$ if $T_a < T_b$
 - $a \rightarrow b$ if $T_a = T_b$ and $Id_a < Id_b$
- The identifier is used to break the tie when two events from different processes have the same clock value.
- Totally ordered logical clock imposes an order between each pair of events in the system.



Use ID if Same. Time stamp.

Assume $\underline{P_1} < \underline{P_2}$.

$A \rightarrow B$ Since $(0, P_1) < (0, P_2)$.

Total
order
Time
stamp